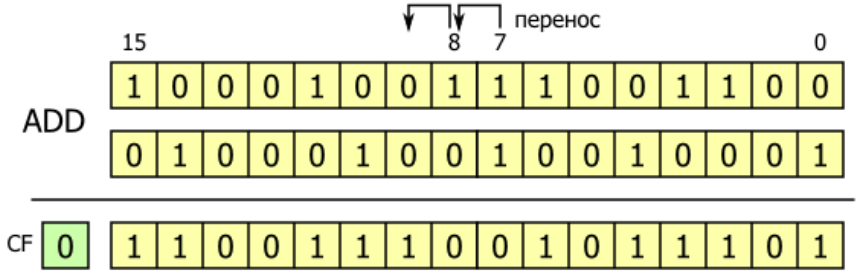
Сложение и вычитание с переносом

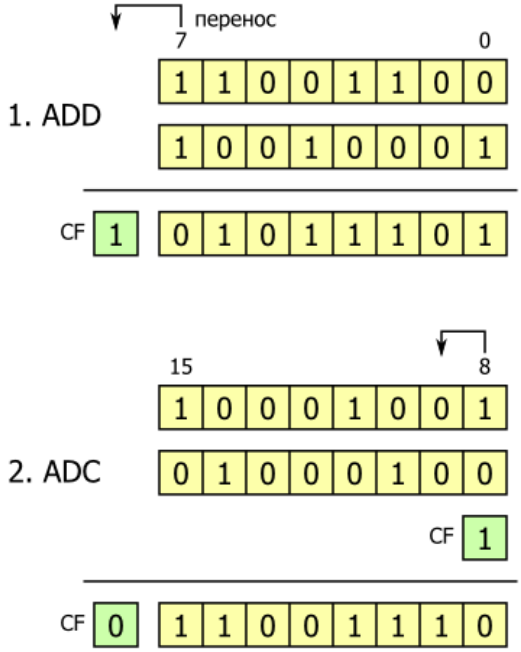
В системе команд процессоров x86 имеются специальные команды сложения и вычитания с учётом флага переноса (*CF*). Для сложения с учётом переноса предназначена команда ADC, а для вычитания — SBB. В общем, эти команды работают почти также, как ADD и SUB, единственное отличие в том, что к младшему разряду первого операнда прибавляется или вычитается дополнительно значение флага *CF*.

Зачем нужны такие команды? Они позволяют выполнять сложение и вычитание многобайтных целых чисел, длина которых больше, чем разрядность регистров процессора (в нашем случае 16 бит). Принцип программирования таких операций очень прост — длинные числа складываются (вычитаются) по частям. Младшие разряды складываются(вычитаются) с помощью обычных команд ADD и SUB, а затем последовательно складываются(вычитаются) более старшие части с помощью команд ADC и SBB. Так как эти команды учитывают перенос из старшего разряда, то мы можем быть уверены, что ни один бит не потеряется Этот способ похож на сложение(вычитание) десятичных чисел в столбик.

На следующем рисунке показано сложение двух двоичных чисел командой ADD:



При сложении происходит перенос из 7-го разряда в 8-й, как раз на границе между байтами. Если мы будем складывать эти числа по частям командой ADD, то перенесённый бит потеряется и в результате мы получим ошибку. К счастью, перенос из старшего разряда всегда сохраняется в флаге CF. Чтобы прибавить этот перенесённый бит, достаточно применить команду ADC:



Аналогичная ситуация возникает с вычитанием чисел по частям. Чтобы было совсем понятно, рассмотрим пример программы. Допустим, требуется вычислить значение формулы *k=i+j-n+1*, где переменные *k*, *i*, *j* и *n* являются 32-битными целыми числами без знака. Складывать и вычитать такие числа придётся в два этапа: сначала вычисления будут производиться с младшими словами операндов, а затем со старшими с учётом переноса.

Для прибавления единицы в данном примере нельзя использовать команду INC, так как она не влияет на флаг *CF* и мы можем получить ошибку в результате!

```
use16                   ;Генерировать 16-битный код
org 100h                ;Программа начинается с адреса 100h

    mov ax,word[i]      ;Загружаем младшую часть i в AX
    mov bx,word[i+2]    ;Загружаем старшую часть i в BX

    add ax,word[j]      ;Складываем младшие части i и j
    adc bx,word[j+2]    ;Складываем старшие части i и j

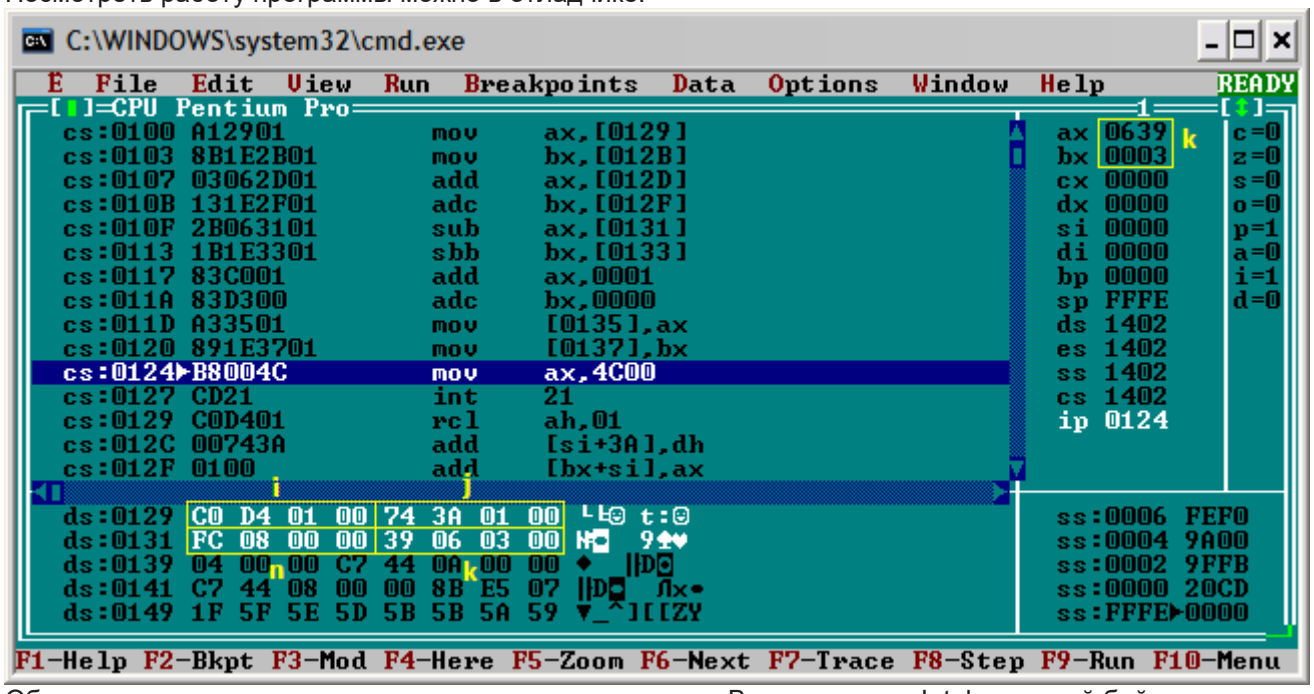
    sub ax,word[n]
    sbb bx,word[n+2]    ;BX:AX = i+j-n

    add ax,1            ;Команда INC здесь не подходит!
    adc bx,0            ;BX:AX = i+j-n+1

    mov word[k],ax      ;\
    mov word[k+2],bx    ;/ Сохраняем результат в k

    mov ax,4C00h        ;\
    int 21h             ;/ Завершение программы
;-------------------------------------------------------
i dd 120000
j dd  80500
n dd   2300
k dd      ?
```

Запись *word[i]* означает, что мы переопределяем размер переменной (она объявлена как DWORD) и обращаемся к младшему слову. Старшее слово расположено в памяти после младшего, поэтому к адресу переменной надо прибавить 2, и соответствующая запись будет иметь вид *word[i+2]*.
Посмотреть работу программы можно в отладчике:



Обратите внимание, как хранятся переменные в памяти. В процессорах Intel младший байт всегда хранится по младшему адресу, поэтому получается, что в окне дампа значения надо читать справа налево. В регистрах же числа записываются в нормальном виде. Сравните, как выглядит одно и то же значение *k* в памяти и в регистрах (старшая часть находится в BX, а младшая — в AX).
Одно из преимуществ ассемблера в том, что на нём можно реализовать работу с собственными форматами чисел, например с очень длинными целыми. А в языках высокого уровня выбор всегда ограничен компилятором. Следующая программа складывает два 7-байтных значения (для разнообразия я использовал только один регистр).

```
use16                   ;Генерировать 16-битный код
org 100h                ;Программа начинается с адреса 100h

    mov ax,word[x]
    add ax,word[y]
```

```
6       mov word[z],ax
7
8       mov ax,word[x+2]
9       adc ax,word[y+2]
10      mov word[z+2],ax
11
12      mov ax,word[x+4]
13      adc ax,word[y+4]
14      mov word[z+4],ax
15
16      mov al,byte[x+6]
17      adc al,byte[y+6]
18      mov byte[z+6],al
19
20      mov ax,4C00h        ;\
21      int 21h             ;/ Завершение программы
22  ;-------------------------------------------------
23  x dd 0xF1111111
24    dw 0xF111
25    db 0x11
26  y dd 0x22222222
27    dw 0x2222
28    db 0x22
29  z rb 7
```

Обращение к старшему байту записывается как *byte[x+6]*. Наверно, вы уже догадались, почему команда MOV не меняет состояние флагов, поэтому её можно ставить между командами сложения.

Упражнение

Напишите программу для вычисления формулы с переносом.

	Задание		Задание
1.	*k=a+b-1-c*	7.	*k=a+4-b+1-c*
2.	*k=a-b+1+c*	8.	*k=a-3+b-1-c*
3.	*k=a-b+4-c+1*	9.	*k=a+b-1-(c+a)*
4.	*k=a-b-1-c*	10.	*k=a+b-(1-c)*
5.	*k=a+b+1+c*	11.	*k=a+b-(1+c)*
6.	*k=a-b+1-c-3*	12.	*k=c+1-(a+b)*

где a,b,c – целые трехбайтные числа без знака.

p/s.

Четные выполняют студенты с четными номерами по списку, не четные соответсвенно с нечетными номерами по спику. Результат работы представить преподавателю двух выражений подробно с пояснениями по флагам. Остальные 4 выражения без коментарий.